Développement coté serveur

Résultats

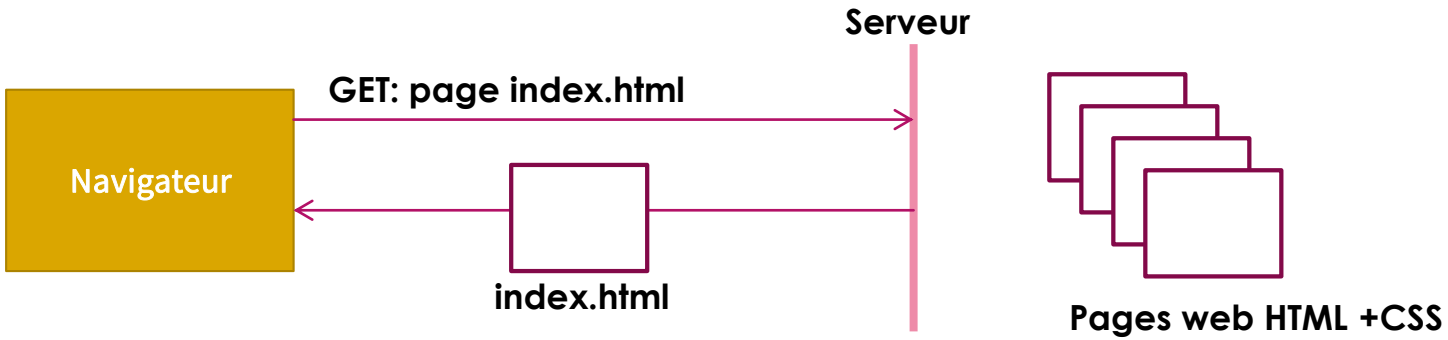
- Maitriser le principe de la programmation web dynamique
- Énumérer les principales technologies du web dynamique
- Maitriser le processus de mise en œuvre d'une application web dynamique avec PHP
- Maitriser la programmation orienté objet avec PHP

Web statique vs web dynamique

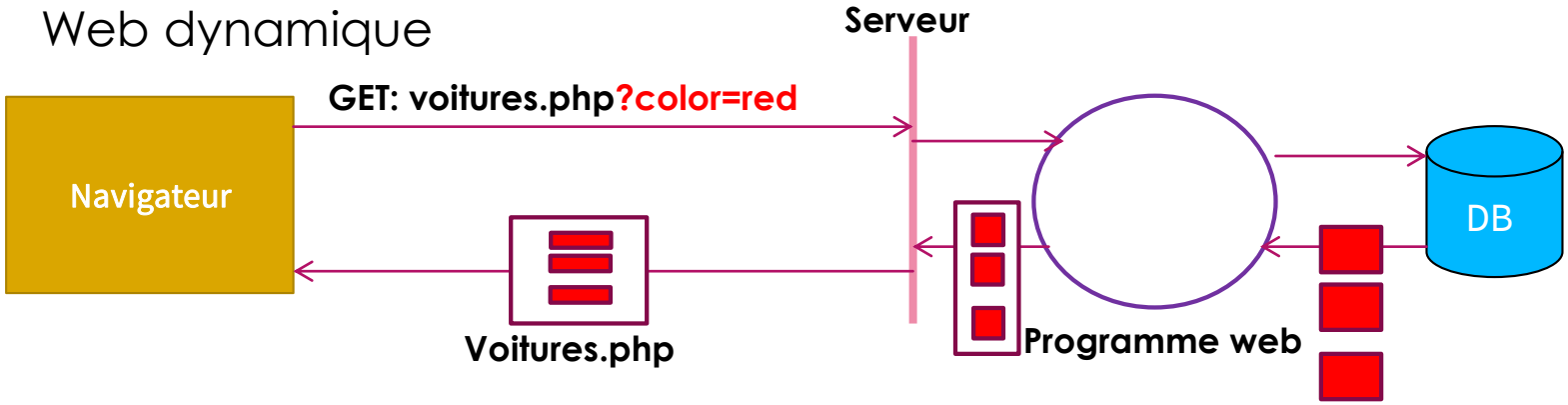
- Il existe deux types de sites web
  - Site web statique
    - Les pages sont statiques, ne changent pas en fonction de l'internaute ou de la requête
    - Développés uniquement avec HTML et CSS
  - Site web dynamique
    - Les pages sont dynamiques, dépendent de l'utilisateur et/ou la requête
    - Basés sur des **langages de programmation** qui génèrent du HTML en fonction de la requête HTTP
    - Les langages de programmation sollicitent très souvent une base de données avant de générer la page web avec les données retournés.
    - Exemple: page web de la liste de voitures bleus

Web statique vs web dynamique

Web statique



Web dynamique



Technologies web dynamique

Langage de programmation	SGBD
PHP (PHP Hypertext Preprocessor)	MySQL
JSP (Java Server Page)	Oracle, MySQL, ou autre
ASP.NET (Microsoft)	Microsoft SQL Server ou autre
…	…

Environnement de travail

- Serveur web:
  - Répondre au requêtes HTTP
  - Gérer les programmes web (PHP par exemple)
  - Exemple : Apache avec le plug-in PHP



- PHP : plug-in pour Apache qui le rend capable de traiter des pages web dynamiques en PHP
- MySQL: Système de gestion de base de données relationnel

WAMP

- Apache Mysql PHP pour Windows
- Plateforme de développement web sous windows
- http://www.wampserver.com/

Premier Script PHP

- Le code PHP s'insère au milieu du code HTML où vous voulez ajouter un contenu dynamique

```
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8" />
        <title>Ma page web</title>
    </head>

    <body>
        <h1>Ma page web</h1>

        <p>
            Bonjour  Insérer le pseudo du visiteur ici  !
        </p>

    </body>
</html>
```

Balise PHP

- Le code PHP est inséré dans une balise PHP pour pouvoir être repérer au moment de la création dynamique de la page

```
<?php
    /* Le code PHP se met ici
    Et ici
    Et encore ici */
?>
```

Afficher du texte: l'instruction echo

- L'instruction "echo" permet d'afficher un message

```
<?php echo "Ceci est du texte"; ?>

<?php echo "Ceci est du <strong>texte</strong>"; ?>

<?php echo "Cette ligne a été écrite \"uniquement\" en PHP."; ?>
```

Caractère d'échappement

Exemple "echo"

```
<!DOCTYPE html>
<html>
    <head>
        <title>Notre première instruction : echo</title>
        <meta charset="utf-8" />
    </head>
    <body>
        <h2>Affichage de texte avec PHP</h2>

        <p>
            Cette ligne a été écrite entièrement en HTML.<br />
            <?php echo "Celle-ci a été écrite entièrement en PHP."; ?>
        </p>
    </body>
</html>
```

Tester une page php

- Une page web qui contient du code php doit être enregistrée avec l'extension ".php"
- Exemple: affichertexte.php
- Doit être enregistrer dans un sous dossier du dossier C:\wamp\www
- Dans notre cas C:\wamp\www\tests
- Dans votre navigateur, tapez l'URL http://localhost/tests/affichertexte.php

Application: afficher la date

```
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8" />
        <title>Ma page web</title>
    </head>
    <body>
        <h1>Ma page web</h1>
        <p>Aujourd'hui nous sommes le <?php echo date('d/m/Y h:i:s'); ?>.</p>
    </body>
</html>
```

Ma page web

Aujourd'hui nous sommes le 26/12/2016 08:12:56.

Générer code HTML



Code PHP d'origine

```
<p>
    Cette ligne a été écrite entièrement en (x)HTML.<br />
    <? echo "Celle-ci a été écrite entièrement en PHP."; ?>
</p>
```

Code généré envoyé au visiteur

```
<p>
    Cette ligne a été écrite entièrement en (x)HTML.<br />
    Celle-ci a été écrite entièrement en PHP.
</p>
```

Les commentaires

Les commentaires mono-lignes

```
<?php
echo "J'habite en Tunisie."; // Cette ligne indique où j'habite

// La ligne suivante indique mon âge
echo "J'ai 21 ans.";
?>
```

Les commentaires multilignes

```
<?php
/* Cette ligne indique
 où j'habite*/
echo "J'habite en Tunisie.";

/* La ligne suivante
indique mon âge*/
echo "J'ai 21 ans.";
?>
```

Les variables

Les variables

- Comme tout langage, PHP manipule des données
- Pour un site dynamique, ces données sont variables
- Ces variables peuvent être de types différents:
  - Chaine de caractère
  - Entier
  - Reel ou flottant
  - Boolean
  - Tableau
  - Objet
  - Resource
  - null

Les variables: nomination

- Chaque variable possède un identifiant particulier, qui commence toujours par le caractère dollar ($) suivi du nom de la variable
- Les règles de création des noms de variable sont les suivantes :
  - Le nom commence par un caractère alphabétique, pris dans les ensembles [a-z], [AZ] ou par le caractère de soulignement (_).
  - Les caractères suivants peuvent être les mêmes plus des chiffres.
  - La longueur du nom n'est pas limitée, mais il convient d'être raisonnable
  - La déclaration des variables n'est pas obligatoire en début de script

Les variables: exemples

- Les noms de variables suivants sont légaux :
  - **$mavar**
  - **$_mavar**
  - **$mavar2**
  - **$M1**
  - **$_123**
- Les suivants sont illégaux :
  - **$5mavar**
  - **$*mavar**
  - **$mavar+**
  - **mavar**

Exercice

- Parmi les variables suivantes, lesquelles ont un nom valide :
  - Mavar
  - $mavar
  - $var5
  - $_mavar
  - $_5var
  - $__élément1
  - $hotel4*

Affectation par valeur et par référence

- L'affectation consiste à donner une valeur à une variable
- Dans PHP, vous pouvez affecter une variable par valeur ou par référence
- Affectation par valeur:
  - $mavar=expression
  - $mavar=$mavar1
  - $mavar=75
  - $mavar=mysql_connect($a,$b,$c);
- Affectation par référence
  - $mavar1="Tunis";
  - $mavar2="Sidi Bouzid";
  - $mavar1=&$mavar2;
  - $mavar2="Sfax";

Affectation par valeur et par référence: Exemple

```php
<?php
//Affectation par valeur de $mavar1 et $mavar2
$mavar1="Tunis"; echo "\$mavar1= ",$mavar1,"<br />";
$mavar2="Sidi Bouzid"; echo "\$mavar2= ",$mavar2,"<br />";
//Affectation par référence de $mavar2
$mavar2 = &$mavar1; echo "Affectation par référence de \$mavar2 <br />";
echo "\$mavar1= ",$mavar1,"<br />" ; echo "\$mavar2= ",$mavar2,"<br />";
echo "modification de \$mavar1 <br />";
$mavar1="Sfax"; echo "\$mavar1= ",$mavar1,"<br />";
echo "\$mavar2= ",$mavar2,"<br />"; echo "modification de \$mavar2 <br />";
$mavar2="Gabes"; echo "\$mavar1= ",$mavar1,"<br />";
echo "\$mavar2= ",$mavar2,"<br />";
?>
```

Résultat

Déterminer le type d'une variable

- Avant de manipuler des variables, en utilisant, par exemple, des opérateurs, il peut être utile de connaître leur type.
- Cela permet de s'assurer que le résultat obtenu est conforme à ce qui est attendu
- La principale fonction permettant de déterminer le type d'une valeur est **gettype()**, dont la syntaxe est la suivante :
  - string gettype($mavar): retourne le nom du type

Déterminer le type d'une variable

- Les fonctions suivantes permettent de vérifier si une variable est d'un type précis :
  - is_integer($var) ou is_int($var)
  - is_double($var)
  - is_string($var)
  - is_bool($var)
  - is_array($var)
  - is_object($var)
  - is_resource($var)
  - is_null($var)
- Elles retournent la valeur booléenne TRUE si la variable est du type recherché et FALSE dans le cas contraire.

Exercice

- Interpréter le résultat de code PHP suivant:

```php
<?php
$var = 73;
if(is_int($var))
{ $var++;
echo "La variable vaut $var <br/>";
}
$var2="Bonjour ";
if(is_string($var2))
{ $var2.=" à tous!";
    echo $var2;
} // Affiche :Bonjour à tous !
?>
```

Résultat

Les opérateurs numériques

Opérateur	Description
+	Addition
-	Soustraction
*	Multiplication

**	Puissance
	$a=3;
	echo $a**2; //Affiche 9
	echo $a**2**4; //Affiche 43046721 soit 3**(2**4) ou 316

/	Division
%	Modulo: reste de division
--	Décrémentation
++	incrémentation

Type boolean

- True ou False
- Exemple:

```
<?php
$a=80;
$b= ($a<95);
echo "\$b vaut ",$b,"<br />";
?>
```

affiche $b vaut 1.

- PHP assimile en interne la valeur TRUE à 1 et la valeur FALSE à 0

Opérateurs de comparaison

Opérateur	Description
==	égalité
!=	inégalité
===	Identité des valeurs et des types
!==	Non-identité des deux expression: valeur ou type
<,<=,>,>=	comparaison
<=>	Avec $a<=>$b, retourne -1, 0 ou 1 respectivement si $a<$b, $a=$b ou $a>$b ($a et $b peuvent être des chaînes).

Opérateurs logiques

Opérateur	Description
OR, ||	OU logique
AND, &&	ET logique
!	NON logique

Chaines de caractères

- Les chaînes de caractères sont avec les nombres les types de données les plus manipulés sur un site web
- Toutes les données sont transmises sous forme de chaînes, d'où leur importance.
- Une chaîne de caractères est une suite de caractères alphanumériques contenus entre des guillemets simples (apostrophes) ou doubles. Par exemple :

```
$a = 'PHP7 et MySQL';
$b = "PHP7 et MySQL";
```

Chaines de caractères

- Si les chaînes ne contiennent que des caractères, les deux types de notation sont parfaitement équivalents. Si une chaîne contient une variable, celle-ci est évaluée, et sa valeur incorporée à la chaîne uniquement si vous utilisez des guillemets et non des apostrophes :

```
$a = 'PHP';
$b = 'MySQL';
$c = "PHP et $b";//affiche : PHP et MySQL
$d = 'PHP et $b';
/*affiche PHP et $b car $ et b sont considérés comme
   des caractères sans signification particulière*/
```

Les Tableaux

- Les tableaux représentent un type composé car ils permettent de stocker sous un même nom de variable plusieurs valeurs indépendantes
- Il existe de types de tableaux
  - Indicés
  - Associatifs

Tableaux indicés

```
$tab = array(valeur0,valeur1,…,valeurN);

$tab=[valeur0,valeur1,…,valeurN];

Exemple:
$notes=[10,8,6];

for($i=0;$i<3;$i++){
    echo $notes[$i]."\n";
}
foreach($notes as $note){
    echo $note."\n";
}
```

Tableaux associatifs

- Définit chaque éléments par un couple (clé, valeur)

```
$tab = array("cléA"=>valA, "cléB"=>valB,… "cléZ"=>valZ);

$tab = ["cléA"=>'valA', "cléB"=>'valB',"cléZ"=>'valZ'];

Exemple:
$couleurs=[
    "r" => "rouge",
    "b" => "bleu"
];

foreach($couleurs as $raccouci=>$couleur){
    echo $raccouci."=>".$couleur;
}
```

Exercice

- Afficher les étudiants par classe

```
$etudiants=[
    "MDW2" => ["Mohamed", "Maryam", "Ali"],
    "RSI2" => ["Syrine", "Youssef"]
];

foreach($etudiants as $classe => $liste_etudiants){
    echo "Etudiants de la classe $classe:\n";
    foreach($liste_etudiants as $etudiant){
        echo "$etudiant\n";
    };
};
```

Les structures conditionnelles

La structure if-elseif-else

- Permet d'évaluer deux ou plusieurs conditions exclusives
- Exemple:

```php
<?php
$note=readline("Entrer une note: ");
if($note>10){
    echo "Bravo vous avez la moyenne";
}
elseif($note == 10){
    echo "Vous avez juste la moyenne";
}
else{
    echo "vous n'avez pas la moyenne";
}
```

Exercice: opérateur ===

```php
<?php
$note=readline("Entrer une note: ");
if($note>10){
    echo "Bravo vous avez la moyenne";
}
elseif($note === 10){
    echo "Vous avez juste la moyenne";
}
else{
    echo "vous n'avez pas la moyenne";
}
```

.....
.....
.....

Exercice: opérateur ===

```php
<?php
$note=(int)readline("Entrer une note: ");
if($note>10){
    echo "Bravo vous avez la moyenne";
}
elseif($note === 10){
    echo "Vous avez juste la moyenne";
}
else{
    echo "vous n'avez pas la moyenne";
}
```

.....
.....
.....

Exercice

- Ecrire un script php permettant d'afficher l'état d'un robot:
- 0:en repos, 1: en marche avant, 2 en marche arrière, 3: en rotation à droite, 4 en rotation à gauche
- Discuter la solution:

.....
.....
.....
.....

Structure Switch

```php
<?php
$etat=readline("Entrer l'état du robot: ");

switch($etat){
    case 0 :
        echo "Robot en repot"; break;
    case 1 :
        echo "Robot en marche avant"; break;
    case 2 :
        echo "Robot en marche arrière"; break;
    case 3 :
        echo "Robot en rotation à droite"; break;
    case 4 :
        echo "Robot en rotation à gauche"; break;
    default :
        echo "état indéterminé";
}
```

Exercices (1)

- Donner les valeurs de $x, $y, $z à la fin du script suivant :

```
<?php
$x="PostgreSQL";
$y="MySQL";
$z=&$x;
$x="PHP 7";
$y=&$x;
?>
```

Solution

Exercices (2)

- Donner la valeur de chacune des variables pendant et à la fin du script suivant et vérifier l'évolution du type de ces variables :

```
$x="PHP7";
$a[]=&$x;
$y=" 7e version de PHP";
$z=$y*10;
$x.=$y;
$y*=$z;
$a[0]="MySQL";
```

Solution

Exercice (3)

- Donner la valeur booléenne des variables $a, $b, $c, $d, $e et $f :

```
$a="0";
if($a){echo “a=vrai”;}else{echo
  “a=faux”};
$b="TRUE";
$c=FALSE;
$d=($a OR $b);//tester ||
$e=($a AND $c);//tester &&
$f=($a XOR $b);
```

Solution

Exercice (4)

- Rédiger une expression conditionnelle pour tester si un nombre est à la fois un multiple de 3 et de 5.

Exercice (5)

- Écrire une expression conditionnelle utilisant les variables $age et $sexe dans une instruction if pour sélectionner une personne de sexe féminin dont l'age est compris entre 21 et 40 ans et afficher un message de bienvenue approprié.

Solution

Les chaînes de caractères

Les chaînes de caractères

- Dans un site web, vous manipulez constamment des chaînes de caractères, qui constituent l'essentiel du contenu de la plupart des pages.
- toutes les variables issues de l'envoi d'un formulaire par le poste client sont de type string.

Affichage des chaînes de caractères

```
$nom = "Mohamed";

echo "Bonjour Monsieur ". $nom. " nous sommes le ". date('d’);

print ("Bonjour Monsieur $nom nous sommes le ". date('d') );
```

Longueur d'une chaîne

- Pour déterminer le nombre de caractères d'une chaîne, utilisez la fonction **strlen()**, dont la syntaxe est la suivante :

```
int strlen (string $ch)

$code = "7508" ;
if (strlen($code) != 5) echo "Code erroné !" ;

$lang = "PHP 7" ;
for ($i =0;$i< strlen ($lang);$i++)
{echo "<h1> $lang[$i] </h1>";}
```

Mise en forme des chaînes

```
string strtolower(string $ch)
//retourne la chaîne avec tous les caractères en minuscules.

string strtoupper(string $ch)
retourne la chaîne avec tous les caractères en majuscules.

string ucwords(string $ch)
retourne la chaîne avec toutes les initiales des mots qui la
composent en majuscules.

string ucfirst(string $ch)
retourne la chaîne avec uniquement la première lettre en majuscule.
```

PHP: Orienté objet

Introduction

- PHP est un langage orienté objet à partir de sa version 5
- Une refonte totale a été effectué pour rendre PHP orienté objet

Notion de classe

```php
<?php
class Fruit {

    private $id;
    private $nom;
    private $photo;
    private $prix;

    public function __construct($id,$nom,$photo,$prix){
        $this->id=$id;
        $this->nom=$nom;
        $this->photo=$photo;
        $this->prix=$prix;
    }

    public function getNom(){
        return $this->nom;
    }
    public function setNom($nom){
        $this->nom=$nom;
    }
    . . .
}?>
```

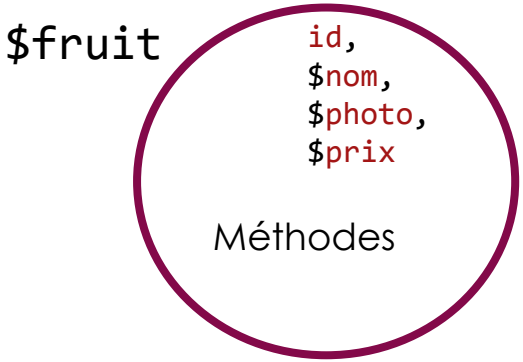
Attributs: représentent l'état de l'objet

Constructeur: état initial

Méthodes: getters et setters

Instanciation

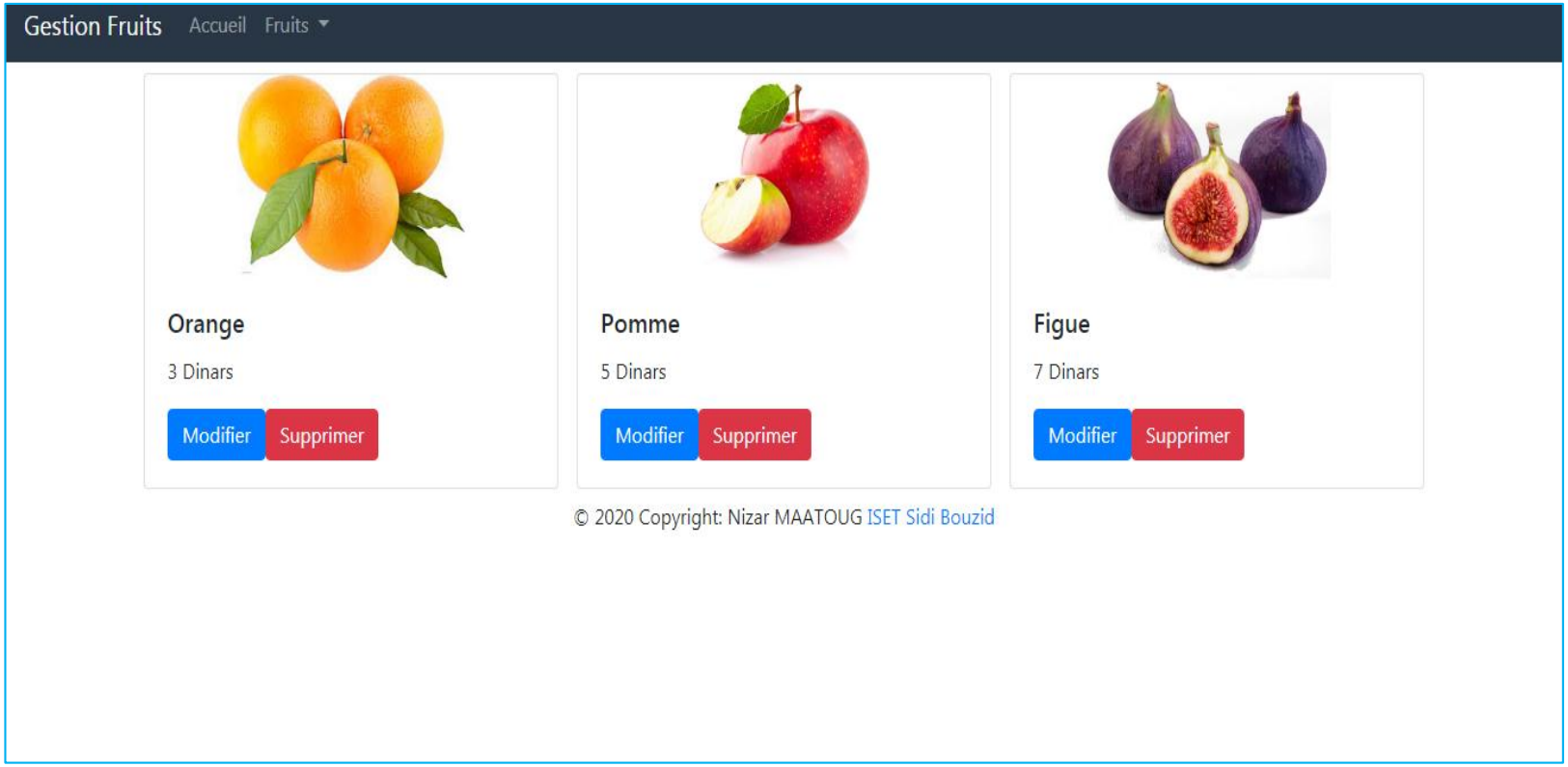
```
$fruit = new Fruit($id,$nom,$photo,$prix);
```



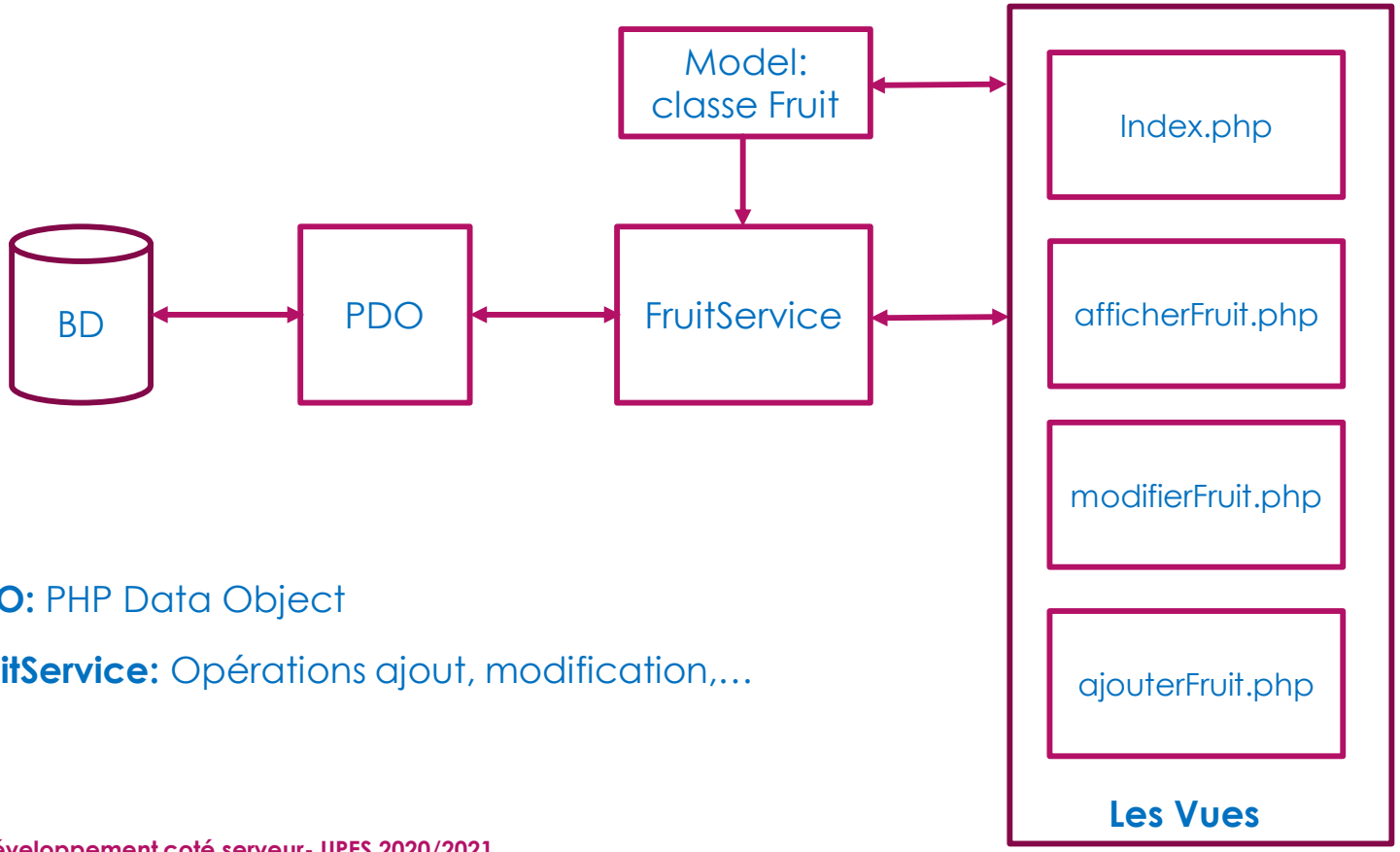
Application: Gestion Fruits

- On souhaite développer une application de gestion de fruits
- Un fruit est caractérisé par:
  - Un identifiant
  - Un nom
  - Un prix
  - Une photo
- L'application permet de:
  - Afficher la liste des fruits
  - Ajouter un fruit
  - Modifier un fruit
  - Supprimer un fruit

Démo


Gestion Fruits
Accueil
Fruits
Orange
3 Dinars
Modifier
Supprimer
Pomme
5 Dinars
Modifier
Supprimer
Figue
7 Dinars
Modifier
Supprimer
© 2020 Copyright: Nizar MAATOUG ISET Sidi Bouzid

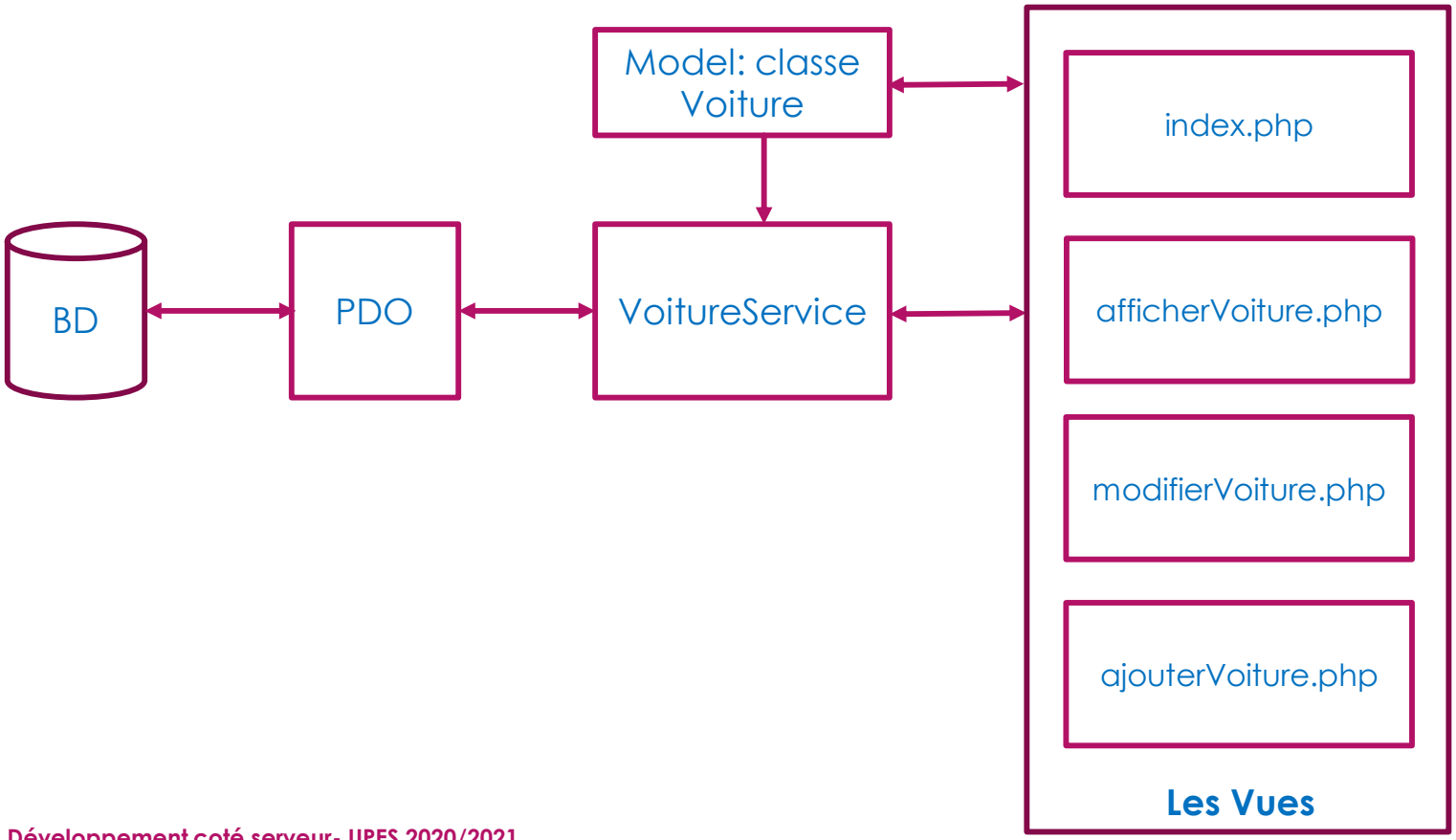
Architecture



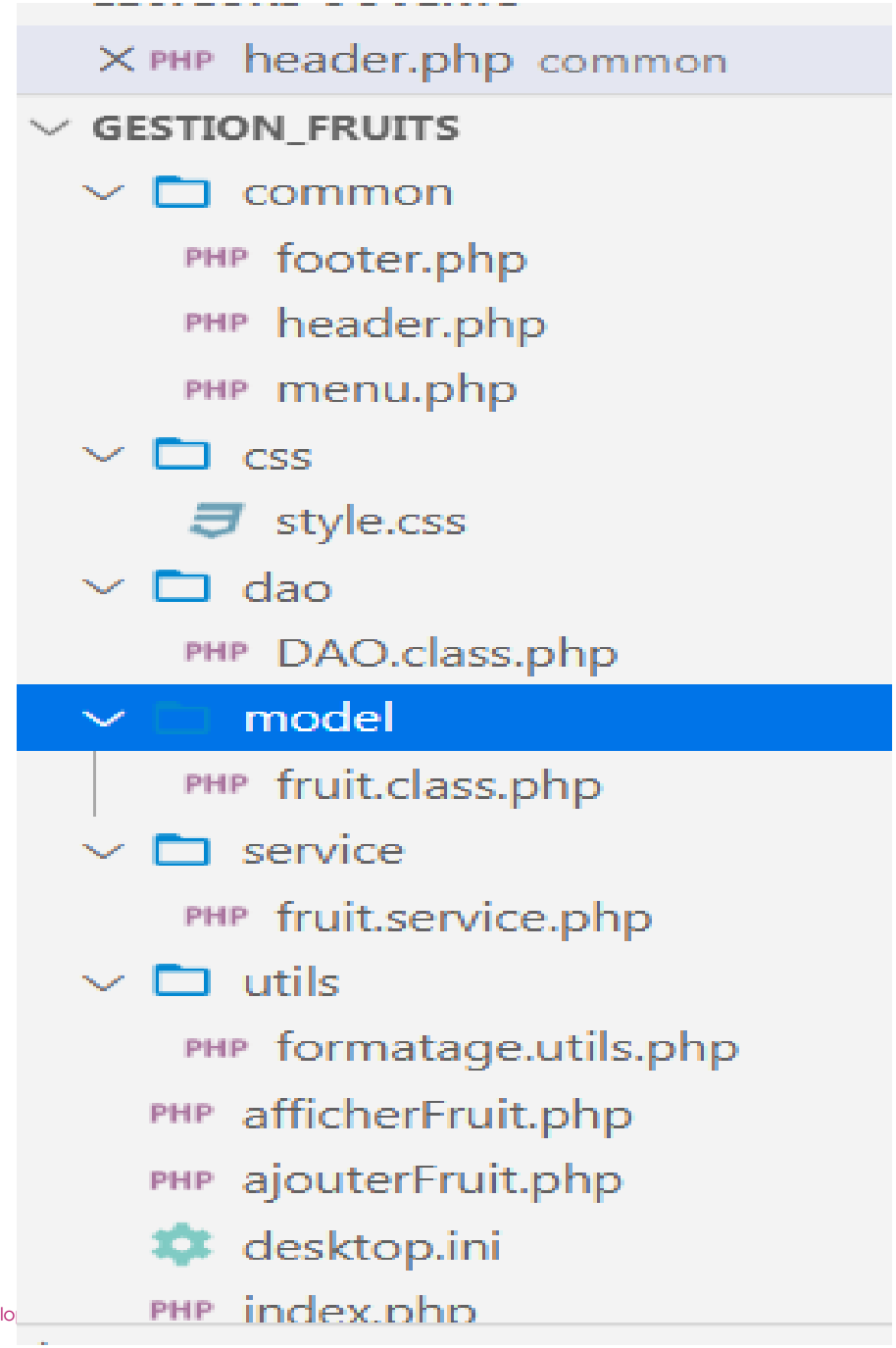
PDO: PHP Data Object

FruitService: Opérations ajout, modification,…

Architecture



Structure du projet



Accès aux données: PHP Data Object

```php
<?php
class DAO {

    private const HOST_NAME = "localhost";
    private const DB_NAME = "fruitsdb";
    private const USER_NAME = "root";
    private const PWD = "";

    private static $PDOinstance = null;

    public static function getPDO(){
        if(is_null(self::$PDOinstance)){
            try {
                $connexion = 'mysql:host='.self::HOST_NAME.';dbname='.self::DB_NAME;
                self::$PDOinstance = new PDO($connexion,self::USER_NAME,self::PWD,
                array(PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION));
            } catch(PDOException $e){
                $message = "erreur de connexion à la DB". $e->getMessage();
                die($message);
            }
            self::$PDOinstance->exec("SET CHARACTER SET UTF8");
        }
        return self::$PDOinstance;
    }
}?>
```

Singleton: une seule instance

FruitService: manipulation des données

```php
<?php

    require_once("dao/DAO.class.php");
    require_once("model/fruit.class.php");

    class FruitService{

        //opérations d'ajout, recherche, modification, suppression



    }

?>
```

FruitService: recherche liste fruits

```
public static function getFruits(){

            $pdo=DAO::getPDO();
            $stmt = $pdo-
>prepare("Select f.id as id, f.nom as nom, f.prix as prix, f.photo as photo from fruit f");
            $stmt->execute();
            $fruits_data = $stmt->fetchAll();
            $fruits=[];
            foreach ($fruits_data as $fruit){
                $fruits[] = new Fruit($fruit['id'],$fruit['nom'],$fruit['photo'],$fruit['prix']);
            }
            return $fruits;
        }
```

FruitService: Recherche d'un fruit

```
public static function getFruit($id) {
         $pdo=DAO::getPDO();
         $stmt = $pdo-
>prepare("Select f.id as id, f.nom as nom, f.prix as prix, f.photo as photo from fruit f where f.id=:id");

         $stmt->bindValue(":id", $id, PDO::PARAM_STR);

         $stmt->execute();

         $fruit_data = $stmt->fetch();

         $fruit=new Fruit($fruit_data['id'],$fruit_data['nom'],$fruit_data['photo'],$fruit_data['prix']);

            return $fruit;

        }
```

FruitService: ajout fruit

```
public static function ajouterFruit($fruit){

            $pdo = DAO::getPDO();

            $req = "insert into fruit values (:id,:nom,:photo,:prix)";

            $stmt = $pdo->prepare($req);

            $stmt->bindValue(":id", null, PDO::PARAM_STR);
            $stmt->bindValue(":nom", $fruit->getNom(), PDO::PARAM_STR);
            $stmt->bindValue(":prix", $fruit->getPrix(), PDO::PARAM_INT);
            $stmt->bindValue(":photo", $fruit->getPhoto(), PDO::PARAM_STR);

            try{
                return $stmt->execute();
            } catch (PDOException $e){
                echo "Erreur : ". $e->getMessage();
                return false;
            }
}
```

FruitService: mise à jour fruit

```
public static function modifierFruit($fruit){

            $pdo = DAO::getPDO();

            $req = "update fruit set nom=:nom,prix=:prix,photo=:photo where id = :id";

            $stmt = $pdo->prepare($req);

            $stmt->bindValue(":nom", $fruit->getNom(), PDO::PARAM_STR);
            $stmt->bindValue(":prix", $fruit->getPrix(), PDO::PARAM_INT);
            $stmt->bindValue(":photo", $fruit->getPhoto(), PDO::PARAM_STR);
            $stmt->bindValue(":id", $fruit->getId(), PDO::PARAM_INT);

            try{
                return $stmt->execute();
            } catch (PDOException $e){
                echo "Erreur : ". $e->getMessage();
                return false;
            }
}
```

FruitService: supprimer fruit

```
public static function deleteFruit($id){

            $pdo = DAO::getPDO();

            $req = "delete from fruit where id= :id";

            $stmt = $pdo->prepare($req);

            $stmt->bindValue(":id", $id, PDO::PARAM_STR);

            try{
                return $stmt->execute();
            } catch (PDOException $e){
                echo "Erreur : ". $e->getMessage();
                return false;
            }
}
```

Index.php

```
<?php
    include("common/header.php");
    include("common/menu.php");
    require_once("utils/formatage.utils.php");
?>
<div class="container">
    <?php echo utile::gererTitreNiveau1('Développement coté serveur ISET SBZ MDW2 2019-2020'); ?>
    <div class="row">
        <div class="col">
            <h2 class="text-center">Gestion des fruits </h2>
            <div class="mx-auto" style="width:150px"><a class="btn btn-outline-
primary" href="afficherFruit.php" role="button">Gestion des fruits</a></div>
        </div>
  </div>
</div>
<?php
    include("common/footer.php");
?>
```

afficherFruit.php

```php
<?php
    require_once("model/fruit.class.php");
    require_once("service/fruit.service.php");

    include("common/header.php");
    include("common/menu.php");

?>


//body




<?php
    include("common/footer.php");
?>
```

afficherFruit.php: body (1/2)

```
<?php
    echo '<div class="container">';
    echo '<div class="row">';
    foreach(FruitService::getFruits() as $fruit){
        echo '<div class="col-sm p-2">';
            echo '<div class="card" >';
            echo '<img class="card-img-top mx-auto" style="width:200px;height:150px" src="'.$fruit-
>getPhoto().'" alt="Card image cap">';
            echo '<div class="card-body">';
            echo '<h5 class="card-title">'.$fruit->getNom().'</h5>';
            echo '<p class="card-text">'.$fruit->getPrix().' Dinars </p>';
    echo '<a href="modifierFruit.php?id='.$fruit->getId().'" class="btn btn-primary">Modifier</a>';

    echo '<a href="afficherFruit.php?delete='.$fruit->getId().'" class="btn btn-danger">Supprimer</a>';
            echo '</div>';
            echo '</div>';
        echo '</div>';
    }
    echo '</div>';
    echo '</div>';
?>
```

afficherFruit.php: body (2/2)

```php
<?php
    if(isset($_GET['delete'])){
        FruitService::deleteFruit($_GET['delete']);
        header("Location:http://localhost/gestion_fruits/afficherFruit.php");
    }
?>
```